

- command line arguments.....3
- command line help.....3
- startup recipe.....3
- main menu.....4
 - 0: show git status.....4
 - 1: short_git_log.....4
 - 2: show unapplied patches.....4
 - 3: show all patches.....4
 - 4: cherry pick next patch.....4
 - 5: patch_info(sha).....6
 - 6: find commits referring to pattem.....7
 - 7: find commits touching "both modified" files.....7
 - 8: add patch by sha.....8
 - 9: show one git log result8
 - 10: delete one git log result.....8
 - 11: build8
 - 12: cherry-pick --continue8
 - 13: cherry-pick -abort.....8
 - 14: compare patch to upstream.....9
 - 15: show git log completions9
 - 16: add sha list to all patches.....9
 - 17: utility actions9
 - 0: backup branch.....9
 - 1: increment git branch.....9
 - 2: note to log file9
 - 3: save checkpoint.....9
 - 4: shell one_liner.....10
 - 5: enter pdb.....10
 - 6: launch bash.....10
 - 7: show_patch_by_sha.....10
 - 8: start_backport.....10
 - 9: REstart_backport.....10
 - 18: experimental actions.....10

Useful Abstractions.....	10
The Patch List	10
The Pickle File.....	11
The Log File	11
Conflict Resolution Files	11
state['out_dest'].....	11
The State Hash.....	11
The Status Trailer.....	12
Conflict Resolution.....	12
No conflict after cherry-pick	12
Conflict after cherry-pick.....	12
analyze & resolve: arch/mips/kvm/mips.c.....	13
conflict.....	13
relevant patch section	13
relevant section in target file.....	13
resolution notes	14
A more complex conflict resolution example	14
conflict	14
related patch section.....	14
look at the version of svm.c in the patch we're cherry-picking.....	15
Where did the conflicting context come from?(IOW why does HEAD have it but not the patch).....	15
What commits have touched this file between the two highlighted above and HEAD	16
Let's look for commits that remove svm->vcpu.arch.user_fpu = kmem_cache_zalloc(x86_fpu_cache	17
Check occasionally for output from your search	17
The very elided view of that search result is:	17
Let's look a bit deeper at a few commits:	18
e529ef66e6b5, KVM: Move vcpu alloc and init redto common code	18
a9dd6f09d7e5, KVM: x86: Allocate vcpu struct in common x86 code, v5.10-rc1, 10/25/20	19
cherry-pick --continue - after all conflicts resolved.....	20
A detailed look at "commits touching both modified files"	20

command line arguments

command line help

backport.py --help

```
usage: backport.py [-h] [--pickle-num PICKLE_NUM] [--sha-list SHA_LIST]
                  [--log-file LOG_FILE] [--pickle-file PICKLE_FILE]
                  [--pickle-dir PICKLE_DIR] --first-commit FIRST_COMMIT
                  [--no-auto-save NO_AUTO_SAVE]
```

options:

- h, --help show this help message and exit
- pickle-num PICKLE_NUM
checkpoint to load
 - Load state from this file. Used to restart from a known good state
- sha-list SHA_LIST File declaring py array of sha(s)
 - Import a list of SHA ids which becomes(does not add to) the patch list
- log-file LOG_FILE path to log file
- pickle-file PICKLE_FILE
pickle file to load
- pickle-dir PICKLE_DIR
where to store checkpoints
- first-commit FIRST_COMMIT
git log will be first-commit^..HEAD
- no-auto-save NO_AUTO_SAVE
for examining pickles
 - not implemented yet

startup recipe

- cd to the target linux tree where patches will be applied
- backport.py --pickle-dir ~/tmp/infobel_demo --first-commit 9edbacd8f841 --log-file ~/tmp/infobel.log
- Since this is the first time the tool has been started in this directory, there is no pickle file to restore so we see

```
<no logf> loading state: /home/evantass/tmp/infobel_demo/backport.pickle.0
*WARNING* cannot restore pickle file:
/home/evantass/tmp/infobel_demo/backport.pickle.0
Enter 'y' to proceed, else <enter>:
```

- enter 'y'

main menu

0: show git status

- simple git status output

=====

On **branch** trashme

You are **currently cherry-picking** **commit 3438de03e98a**.

(fix conflicts and run "git cherry-pick --**continue**")

(use "git cherry-pick --**skip**" to skip this patch)

(use "git cherry-pick --**abort**" to cancel the cherry-pick operation)

Unmerged paths:

(use "git add <file>..." to mark resolution)

both modified: drivers/crypto/ccp/sp-pci.c

no changes added to commit (use "git add" and/or "git commit -a")

=====

1: short_git_log

- show the commits applied on top of that specified in the --first-commit and HEAD

2: show unapplied patches

@@how_unapplied_patches

>>>>:

0, 3438de03e98a, crypto: ccp - Add support for new CCP/PSP device ID, 5.19.0-1.1, 08/22/22,

unapplied

<<<<:

3: show all patches

- short_git_log followed by unapplied patches

4: cherry pick next patch

This uses git cherry-pick to "apply" the first patch on the list of unapplied patches.

enter choice(<enter> to exit menu): 4

@@apply_next_patch

@@ **cherry picking : 3438de03e98a**, crypto: ccp - Add support for new CCP/PSP device ID, 5.19.0-1.1, 08/22/22, unapplied

* 3438de03e98a

error: could not apply 3438de03e98a... crypto: ccp - Add support for new CCP/PSP device ID

hint: After resolving the conflicts, mark them with

hint: "git add/rm <paths>", then run

hint: "git cherry-pick --continue".

hint: You can instead skip this commit with "git cherry-pick --skip".

hint: To abort and get back to the state before "git cherry-pick",

hint: run "git cherry-pick --abort".

b'Auto-merging drivers/crypto/ccp/sp-pci.c\nCONFLICT (content): Merge conflict in drivers/crypto/ccp/sp-pci.c\n'

On branch trashme

You are currently cherry-picking commit 3438de03e98a.

(fix conflicts and run "git cherry-pick --continue")

(use "git cherry-pick --skip" to skip this patch)

(use "git cherry-pick --abort" to cancel the cherry-pick operation)

Unmerged paths:

(use "git add <file>..." to mark resolution)

both modified: drivers/crypto/ccp/sp-pci.c

no changes added to commit (use "git add" and/or "git commit -a")

9edbacd8f841 Shida Zhang : ext4: fix rec_len verify error: Tue Apr 9 09:51:41 2024 +0000

5819af280752 Chao Yu : ext4: fix to check return value of freeze_bdev() in ext4_shutdown(): Tue Apr 9 09:51:41 2024 +0000

7b2a2c979211 Kemeng Shi : ext4: fix wrong unit use in ext4_mb_new_blocks: Tue Apr 9 09:51:41 2024 +0000

691a3d937fbd Kemeng Shi : ext4: get block from bh in ext4_free_blocks for fast commit replay: Tue Apr 9 09:51:41 2024 +0000

34395d80f79b Kemeng Shi : ext4: fix wrong unit use in ext4_mb_clear_bb: Tue Apr 9 09:51:41 2024 +0000

git blame HEAD^ -- drivers/crypto/ccp/sp-pci.c >

/tmp/cp__files/3438de03e98a/HEAD^.sp-pci.c

git blame HEAD -- drivers/crypto/ccp/sp-pci.c > /tmp/cp__files/3438de03e98a/HEAD.sp-pci.c

git blame 3438de03e98a^ -- drivers/crypto/ccp/sp-pci.c >

/tmp/cp__files/3438de03e98a/3438de03e98a^.sp-pci.c

git blame 3438de03e98a -- drivers/crypto/ccp/sp-pci.c >

/tmp/cp__files/3438de03e98a/3438de03e98a.sp-pci.c

@@save_cp: /home/evantass/tmp/infobel_demo/backport.pickle.5

=====

0: show git status

1: short_git_log

2: show unapplied patches

3: show all patches

4: cherry pick next patch

5: patch_info(sha)

6: find commits referring to pattern

7: find commits touching "both modified" files

8: add patch by sha

9: show one git log result

10: delete one git log result

11: build

12: cherry-pick --continue

13: cherry-pick -abort

14: compare patch to upstream

```
15: show git log completions
16: add sha list to all patches
17: utility actions
18: experimental actions
```

```
1 commits (0 applied)
** working tree UNCLEAN **
** active cherry-pick **
```

```
enter choice(<enter> to exit menu):
```

at this point you should use "0: show git status"

```
..
enter choice(<enter> to exit menu): 0

=====
On branch trashme
You are currently cherry-picking commit 3438de03e98a.
  (fix conflicts and run "git cherry-pick --continue")
  (use "git cherry-pick --skip" to skip this patch)
  (use "git cherry-pick --abort" to cancel the cherry-pick operation)
```

```
Unmerged paths:
  (use "git add <file>..." to mark resolution)
both modified: drivers/crypto/ccp/sp-pci.c
```

```
no changes added to commit (use "git add" and/or "git commit -a")
=====
```

5: patch_info(sha)

Patches are referred to by SHA ids all over because they're unique hash values of a commit and the short form of the hash is a lot less typing than the whole subject line. You'll find this useful when git blame refers you to other commits.

As you can see below it reports the short form of the sha, the commit message subject line, the release the commit first appeared in and that date.

```
enter choice(<enter> to exit menu): 5
@@patch_info
enter sha1(return to exit): 3438de03e98a
3438de03e98a, crypto: ccp - Add support for new CCP/PSP device ID, 5.19.0-1.1,
08/22/22
```

6: find commits referring to pattern

We'll discuss this in greater detail in later sections on conflict resolution, but you can think of "why did adding this patch cause `THING_DEFINE` to be undefined". You'd use this facility to find the patches that introduced and perhaps modified `THING_DEFINE`

enter choice(<enter> to exit menu): 6

@@git_logG_simple

enter 'S' or 'G', <pattern> <enter to return>: G CONFIG_CRYPTO_DEV_SP_PSP

enter release tag for git_log_min_tag: 4.19.91-23.1

enter release tag for git_log_max_tag: 6.6.7-1

launching asynchronous git log task

git_logG_syms/exit

7: find commits touching "both modified" files

This menu item only does everything if you currently have an active commit that has "both modified" files. These are files modified in the commit and by the patches below the one being applied. Git cannot automatically figure out how to resolve the context that it sees in the working directory and the context the patch expects.

Often this command can help you answer "how many commits are touching this section of this file". You need to further answer "how many of them do i need". Then document which you included, which you excluded and why.

At minimum, internal reviewers are going to want to see the list of **commits affecting only the hunk(s) that git is asking you to resolve manually**.

You must be very careful. Only include patches that modify the hunk with differences git asks you to resolve manually.

- The set of commits that modifies a "manual hunk" is much smaller than the set that modifies the whole file.
- Including a **non-needed** (not meeting the above criterion) **patch** very likely risks drawing in **further patches that you don't need**.
- Neglecting to select patches you do need will likely **cause very hard to find and resolve bugs** and a lot of wasted effort.

How to effectively get the "set" of patches reviewed internally

- **in the patch where you're resolving a conflict manually**, add immediately after the subject line, a "requires comment" so you'll get something that looks like:

PATCH: SEV/x86: some patch title

commit <sha of upstream patch you're resolving the conflict for> **upstream**

commit <sha of first pre-requisite identified> **prerequisite**

- o **in the patches you're adding to the patch list**, add immediately after the subject line, a "prcomment" so you'll get something that looks like:

PATCH: SEV/x86: some patch title required by above

commit <sha of upstream patch you're resolving the conflict for> **required_for**

8: add patch by sha

Adds the patch defined by the SHA to the patch list, if it is found.

Does nothing if the sha cannot be found.

9: show one git log result

When you use the git log commands

- 6: find commits referring to pattern
- 7: find commits touching "both modified" files

the work is carried out asynchronously because the command can take a while to run.

The results are saved in the state file and you use this command to display them.

enter choice(<enter> to exit menu): 9

0:S,kmem_cache_zalloc,v5.3,v6.2

enter index <enter to return>: 0

10: delete one git log result

Use this command sparingly. We often want to leave older results around for a while so we can refer back to them.

11: build

- if not set in state,
 - o the command prompts you for your build command(ex: "make -j 300")
 - o saves it in state
 - o and runs it
- if set, the build command is ru

12: cherry-pick --continue

- run "git cherry-pick continue"
- do bookkeeping such as updating the patch state to reflect the downstream sha

13: cherry-pick -abort

Just run "git cherry-pick --abort"

14: compare patch to upstream

Use this command frequently and definitely before requesting internal review or delivering to a customer. What you will see is the difference between the upstream patch and your backported version.

- There should be no surprises.
- You should be able to explain every change
- Reviewers and customers will expect to see these explanations in the patch comments

15: show git log completions

enter choice(<enter> to exit menu): 15

['G,CONFIG_CRYPTODEV_SP_PSP,4.19.91-23.1,6.6.7-1']

this should correlate clearly with what we did above in menu item 6

16: add sha list to all patches

- One SHA1 per line in a text file.
- That's all. No comments or anything else.

enter choice(<enter> to exit menu): 16

@@add_sha_list_to_all_patches

enter the path of a file containing one sha per line <enter> if none
sha_file: /tmp/pl.txt

, 9edbacd8f841, ext4: fix rec_len verify error, , 08/03/23, unapplied
about to reset --hard to this commit
Enter 'y' to proceed, else <enter>:

@@save_cp: /home/evantass/tmp/infobel_demo/backport.pickle.1

- note that the suffix number of the pickle file has incremented. This allows you, in combination with the --pickle-number argument to go back to a previous state if you tried something experimental and it didn't work.

17: utility actions

0: backup branch

- prompt for and save the command to push your working branch to a remote
 - don't try to push to linux stable or any upstream tree
- push the contents of your local branch to the remote repo

1: increment git branch

- Use this if your about to do something that makes you nervous so you can go back to the checkpoint before you did the dangerous thing.

2: note to log file

- Add a line to the log file to explain what you just did or what you're about to do.

3: save checkpoint

- Write the pickle file

4: shell one_liner

- A sometimes useful way to get a quick task done without having to open a shell window.

5: enter pdb

Enter the python debugger which allows you to

- examine and modify state
- set breakpoints, ...

6: launch bash

launch a shell

7: show_patch_by_sha

Show the content of a patch for which you know the SHA.

8: start_backport

- do some internal housekeeping
- reset the branch to the commit specified via --first-commit which has been saved in state

9: REstart_backport

- abort any active cherry pick
- increment the branch number
 - incremented the same way as pickle file names.
- call start_backport()

18: experimental actions

- For now, don't use these

Useful Abstractions

The Patch List

- The patch list is a list of "patch objects"
- The patch list contains applied and unapplied patches
- An entry in the patch list is guaranteed unique by SHA
- **The patch list is sorted by the position that a patch has in the list of patches tagged with tag of the highest release tag of all patches in the patch list** (read that a few times)
 - For example if the patch list looks like: P100, P250, P300 (where the numbers are the position referred to above)
 - and you discover you need to add P75
 - the resulting patch list will look like: P75, P100, P250, P300
 - P75 will be the "next to apply"
 - the branch will be reset to before P75
 - you'll have to reapply P75, P100, P250 and P300
 - the conflicts you had previously for P75, P100, P250 and P300 will be saved and, **if the conflicts after applying P75 are the same as they were previously, then your previous resolution will be re-used** and you won't have to do it again.
- **This is the single most important abstraction of the tool.**

The Pickle File

- **Every time you "do something important"**, which usually means you did something that changes the state of an individual patch object(ex: unapplied -> applied) or added a patch to the patch list
 - the number at the end of the pickle file is incremented
 - the contents of the state object are written to the pickle file
- **The pickle file is read when backport.py starts so**
 - you can continue where you left off before even if the system has rebooted
 - you can use the --pickle-num to go back to a previous state
 - this is not a trivial operation
 - you may have to use git commands to reset the HEAD of your branch so that nothing in the "new" patch list which is unapplied is already applied in your working directory
 - do this with caution.
- For large projects you may want to delete some of the older pickle files to free disk space.

The Log File

- The tool records almost everything it does and the results
- This is useful when you have to **go back and answer questions** like
 - "when and why was this patch added to the backport?"
 - "what pickle file contains the last verifiably good patch list before I tried some speculative actions which didn't work out so well?"
- The utility menu allows you to add notes to the log file

Conflict Resolution Files

state['out_dest']

- the landing place for conflict resolution files
- each patch that is cherry-picked and has conflicts will cause a directory to be created here including
 - **patch**
 - **blame.***, **prev.blame.***: output of git blame on HEAD and HEAD^
 - HEAD versions of conflicted files

The State Hash

The state hash contains lots of things

- The most important is the **patch list**.
- Repetitive things are "remembered" like
 - the kernel build command you use for this project
 - the command you use to back up your local git tree to an internal github

You can get to the python debugger(pdb) via the utility submenu and alter things in state but it's not advisable unless you're really sure you know what you're doing and you've explicitly saved a checkpoint to a pickle file(again via the utility submenu) before doing so.

The Status Trailer

```
20 commits (0 applied)
** working tree UNCLEAN **
** active cherry-pick **
```

Conflict Resolution

No conflict after cherry-pick

From git status, we see:

```
=====
On branch 10cent.x2avic.5.4.32_v3.8
You are currently cherry-picking commit 4543bdc08857.
(all conflicts fixed: run "git cherry-pick --continue")
(use "git cherry-pick --skip" to skip this patch)
(use "git cherry-pick --abort" to cancel the cherry-pick operation)
```

Changes to be committed:

```
modified: arch/mips/kvm/mips.c
modified: arch/powerpc/kvm/powerpc.c
modified: arch/s390/kvm/kvm-s390.c
modified: arch/x86/kvm/x86.c
modified: include/linux/kvm_host.h
modified: virt/kvm/arm/arm.c
modified: virt/kvm/kvm_main.c
```

Conflict after cherry-pick

```
error: could not apply 4543bdc08857... KVM: Introduce kvm_vcpu_destroy()
hint: After resolving the conflicts, mark them with
hint: "git add/rm <pathspec>", then run
hint: "git cherry-pick --continue".
hint: You can instead skip this commit with "git cherry-pick --skip".
hint: To abort and get back to the state before "git cherry-pick",
hint: run "git cherry-pick --abort".
b'Auto-merging arch/mips/kvm/mips.c\nCONFLICT (content): Merge conflict in
arch/mips/kvm/mips.c\nAuto-merging arch/powerpc/kvm/powerpc.c\nCONFLICT (content):
Merge conflict in arch/powerpc/kvm/powerpc.c\nAuto-merging arch/s390/kvm/kvm-
s390.c\nAuto-merging arch/x86/kvm/x86.c\nCONFLICT (content): Merge conflict in
arch/x86/kvm/x86.c\nAuto-merging include/linux/kvm_host.h\nAuto-merging
virt/kvm/arm/arm.c\nCONFLICT (content): Merge conflict in virt/kvm/arm/arm.c\nAuto-
merging virt/kvm/kvm_main.c\n'
On branch 10cent.x2avic.5.4.32_v3.8
You are currently cherry-picking commit 4543bdc08857.
(fix conflicts and run "git cherry-pick --continue")
(use "git cherry-pick --skip" to skip this patch)
(use "git cherry-pick --abort" to cancel the cherry-pick operation)
```

Changes to be committed:

modified: arch/s390/kvm/kvm-s390.c

modified: include/linux/kvm_host.h

modified: virt/kvm/kvm_main.c

Unmerged paths:

(use "git add <file>..." to mark resolution)

both modified: arch/mips/kvm/mips.c

both modified: arch/powerpc/kvm/powerpc.c

both modified: arch/x86/kvm/x86.c

both modified: virt/kvm/arm/arm.c

analyze & resolve: arch/mips/kvm/mips.c

conflict

```
void kvm_mips_free_vcpus(struct kvm *kvm)
{
    unsigned int i;
    struct kvm_vcpu *vcpu;

    kvm_for_each_vcpu(i, vcpu, kvm) {
<<<<<< HEAD
        kvm_arch_vcpu_free(vcpu);
=====
        kvm_vcpu_destroy(vcpu);
>>>>>> 4543bdc08857 (KVM: Introduce kvm_vcpu_destroy())
    }

    mutex_lock(&kvm->lock);
```

:

relevant patch section

--- a/arch/mips/kvm/mips.c

+++ b/arch/mips/kvm/mips.c

@@ -156,7 +156,7 @@ void kvm_mips_free_vcpus(struct kvm *kvm)
struct kvm_vcpu *vcpu;

kvm_for_each_vcpu(i, vcpu, kvm) {
- kvm_arch_vcpu_destroy(vcpu);
+ kvm_vcpu_destroy(vcpu);
}

mutex_lock(&kvm->lock);

relevant section in target file

```
void kvm_mips_free_vcpus(struct kvm *kvm)
{
    unsigned int i;
    struct kvm_vcpu *vcpu;

    kvm_for_each_vcpu(i, vcpu, kvm) {
        kvm_arch_vcpu_free(vcpu);
    }
```

```
mutex_lock(&kvm->lock);
```

resolution notes

- this should not have been a conflict
- accept the green line
- handled the same way:
 - powerpc.c
 - x86.c
 - arm.c

A more complex conflict resolution example
conflict

```
<<<<<<< HEAD
svm = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL_ACCOUNT);
if (!svm) {
    err = -ENOMEM;
    goto out;
}

svm->vcpu.arch.user_fpu = kmem_cache_zalloc(x86_fpu_cache,
                                           GFP_KERNEL_ACCOUNT);
if (!svm->vcpu.arch.user_fpu) {
    printk(KERN_ERR "kvm: failed to allocate kvm userspace's fpu\n");
    err = -ENOMEM;
    goto free_partial_svm;
}

svm->vcpu.arch.guest_fpu = kmem_cache_zalloc(x86_fpu_cache,
                                           GFP_KERNEL_ACCOUNT);
if (!svm->vcpu.arch.guest_fpu) {
    printk(KERN_ERR "kvm: failed to allocate vcpu's fpu\n");
    err = -ENOMEM;
    goto free_user_fpu;
}

err = kvm_vcpu_init(&svm->vcpu, kvm, id);
if (err)
    goto free_svm;

=====
>>>>>>> 987b2594ed5d (KVM: x86: Move kvm_vcpu_init() invocation to common code)
```

related patch section

```
@@ -2200,14 +2199,10 @@ static int svm_create_vcpu(struct kvm *kvm, struct kvm_vcpu
*vcpu,
BUILD_BUG_ON(offsetof(struct vcpu_svm, vcpu) != 0);
svm = to_svm(vcpu);

-     err = kvm_vcpu_init(vcpu, kvm, id);
-     if (err)
-         return err;
-
err = -ENOMEM;
```

```
page = alloc_page(GFP_KERNEL_ACCOUNT);
if (!page)
-         goto uninit;
+         goto out;

msrpm_pages = alloc_pages(GFP_KERNEL_ACCOUNT, MSRPM_ALLOC_ORDER);
if (!msrpm_pages)
```

look at the version of svm.c in the patch we're cherry-picking

```
static int svm_create_vcpu(struct kvm_vcpu *vcpu) // note: not kvm_vcpu_create
{
    struct vcpu_svm *svm;
    struct page *page;
    struct page *msrpm_pages;
    struct page *hsave_page;
    struct page *nested_msrpm_pages;
    int err;

    BUILD_BUG_ON(offsetof(struct vcpu_svm, vcpu) != 0);
    svm = to_svm(vcpu);

    err = -ENOMEM;
    page = alloc_page(GFP_KERNEL_ACCOUNT);
    if (!page)
        goto out;

    msrpm_pages = alloc_pages(GFP_KERNEL_ACCOUNT, MSRPM_ALLOC_ORDER);
    if (!msrpm_pages)
        goto free_page1;

    nested_msrpm_pages = alloc_pages(GFP_KERNEL_ACCOUNT, MSRPM_ALLOC_ORDER);
    if (!nested_msrpm_pages)
        goto free_page2;

    hsave_page = alloc_page(GFP_KERNEL_ACCOUNT);
    if (!hsave_page)
        goto free_page3;

    err = avic_init_vcpu(svm);
```

Where did the conflicting context come from?(IOW why does HEAD have it but not the patch)

- It's clear that there is much context being deleted in the conflict resolution above
 - A question that must be answered is "What patches deleted that context so it wasn't present in the commit we're cherry-picking".
 - from **git blame HEAD -- arch/x86/kvm/svm.c** we can see:

```
12b58f4ed2a1b arch/x86/kvm/svm.c (Sean Christopherson    2019-08-15 10:22:37 -
0700 2187)      BUILD_BUG_ON_MSG(offsetof(struct vcpu_svm, vcpu) != 0,
```

```

12b58f4ed2a1b arch/x86/kvm/svm.c (Sean Christopherson    2019-08-15 10:22:37 -
0700 2188)          "struct kvm_vcpu must be at offset 0 for arch usercopy region");
12b58f4ed2a1b arch/x86/kvm/svm.c (Sean Christopherson    2019-08-15 10:22:37 -
0700 2189)
1ec696470c860 arch/x86/kvm/svm.c (Ben Gardon            2019-02-11 11:02:51 -0800
2190)   svm = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL_ACCOUNT);
fb3f0f51d92d1 drivers/kvm/svm.c (Rusty Russell          2007-07-27 17:16:56 +1000
2191)   if (!svm) {
fb3f0f51d92d1 drivers/kvm/svm.c (Rusty Russell          2007-07-27 17:16:56 +1000
2192)       err = -ENOMEM;
fb3f0f51d92d1 drivers/kvm/svm.c (Rusty Russell          2007-07-27 17:16:56 +1000
2193)       goto out;
fb3f0f51d92d1 drivers/kvm/svm.c (Rusty Russell          2007-07-27 17:16:56 +1000
2194)   }
fb3f0f51d92d1 drivers/kvm/svm.c (Rusty Russell          2007-07-27 17:16:56 +1000
2195)
d9a710e5fc494 arch/x86/kvm/svm.c (Wanpeng Li            2019-07-22 12:26:21 +0800
2196)   svm->vcpu.arch.user_fpu = kmem_cache_zalloc(x86_fpu_cache,
d9a710e5fc494 arch/x86/kvm/svm.c (Wanpeng Li            2019-07-22 12:26:21
fb3f0f51d92d1 drivers/kvm/svm.c (Rusty Russell          2007-07-27 17:16:56 +1000
2192)       err = -ENOMEM;
fb3f0f51d92d1 drivers/kvm/svm.c (Rusty Russell          2007-07-27 17:16:56 +1000
2193)       goto out;
fb3f0f51d92d1 drivers/kvm/svm.c (Rusty Russell          2007-07-27 17:16:56 +1000
2194)   }
fb3f0f51d92d1 drivers/kvm/svm.c (Rusty Russell          2007-07-27 17:16:56 +1000
2195)
d9a710e5fc494 arch/x86/kvm/svm.c (Wanpeng Li            2019-07-22 12:26:21 +0800
2196)   svm->vcpu.arch.user_fpu = kmem_cache_zalloc(x86_fpu_cache,
d9a710e5fc494 arch/x86/kvm/svm.c (Wanpeng Li            2019-07-22 12:26:21 +0800
2197)       GFP_KERNEL_ACCOUNT);
d9a710e5fc494 arch/x86/kvm/svm.c (Wanpeng Li

```

What commits have touched this file between the two highlighted above and HEAD

=====

...

7: find commits touching "both modified" files

...

>>>>>:

0, 50896de4be7e, KVM: x86: always expose VIRT_SSBD to guests, v5.10-rc1, 10/25/20,
unapplied

1, 05402f64544f, x86: KVM: svm: don't pretend to advance RIP in case wrmsr_interception()
results in #GP, v5.10-rc1, 10/25/20, unapplied

...

48, fc6e2a1845ab, KVM: x86: Move FPU allocation to common x86 code, v5.10-rc1, 10/25/20,
unapplied

49, 987b2594ed5d, KVM: x86: Move kvm_vcpu_init() invocation to common code, v5.10-rc1,
10/25/20, cherry-pick active

<<<<<:

50 patches is quite a lot to review before we know we need to, so answer '!' at each prompt until we get back to the menu and we'll pick a smaller hammer.

Let's look for commits that remove `svm->vcpu.arch.user_fpu =`

`kmem_cache_zalloc(x86_fpu_cache`

=====

...

6: find commits referring to pattern

...

Check occasionally for output from your search

- **Several variables can influence how long these searches take**
 - the span between the minimum and maximum commits(usually release tags)
 - searching from v2.6 through v6.9 will take longer than v6.8 - v6.9
 - a sensible default set is
 - min = <the release tag you're backporting to>
 - max = the commit id of the last commit to be backported
 - these variables are defined in the **state** object keys 'git_log_min_tag', 'git_log_max_tag'
 - there is no interface to set them other than going into pdb and evaluating expressions
 - this isn't too burdensome since it's usually "set & forget" at the beginning of a backporting project
 - The difference between 'git log -S' and 'git log -G' is that 'S' only reports changes in the number of references to the pattern between a commit and it's predecessor
 - if there was a line "count = 0" that was changed to "count = 1", 'G' would find it but 'S' would not
 - 'S' is faster

=====

...

9: show one git log result

.

enter choice(<enter> to exit menu): 9

0:S,kmem_cache_zalloc,v5.3,v6.2

enter index <enter to return>: 0

The very elided view of that search result is:

- recall, from above, this snippet of the context that our analysis determined must have been deleted by some commit preceding the one we're trying to cherry-pick

conflict

```

<<<<<< HEAD
svm = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL_ACCOUNT);
if (!svm) {
    err = -ENOMEM;
    goto out;
}

```

-
- Given that, we look at the whole git log result and look for patches that
 - refer to **kmem_cache_alloc** and **kvm_cpu_cache**.
 - delete such lines,
- Pruning the search results by those criteria we reduce 300 lines of diff to a few candidate patches.

== , **e529ef66e6b5, KVM: Move vcpu alloc and init redto common code**, v5.10-rc1, 10/25/20, unapplied ==

```

- struct kvm_vcpu *vcpu = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL);
- vcpu = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL);
- vcpu = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL);
- vcpu = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL_ACCOUNT);
- vcpu = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL);
+ vcpu = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL);

```

== , **c50bfbd38ec, KVM: PPC: Allocate vcpu struct in common PPC code**, v5.10-rc1, 10/25/20, unapplied ==

```

- vcpu = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL);
- vcpu = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL);
- vcpu_e500 = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL);
- vcpu_e500 = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL);
+ vcpu = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL);

```

== , **a9dd6f09d7e5, KVM: x86: Allocate vcpu struct in common x86 code**, v5.10-rc1, 10/25/20, unapplied ==

```

- svm = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL_ACCOUNT);
vcpu->arch.user_fpu = kmem_cache_zalloc(x86_fpu_cache,
vcpu->arch.guest_fpu = kmem_cache_zalloc(x86_fpu_cache,
- vmx = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL_ACCOUNT);
vcpu->arch.user_fpu = kmem_cache_zalloc(x86_fpu_cache,
vcpu->arch.guest_fpu = kmem_cache_zalloc(x86_fpu_cache,
+ vcpu = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL_ACCOUNT);

```

Let's look a bit deeper at a few commits:

- recall, from above, this snippet of the context that our analysis determined must have been

e529ef66e6b5, KVM: Move vcpu alloc and init redto common code

```

int kvm_arch_vcpu_runnable(struct kvm_vcpu *vcpu)
diff --git a/arch/x86/kvm/x86.c b/arch/x86/kvm/x86.c
index 42b9149f6b40..7bbde6f658bf 100644
--- a/arch/x86/kvm/x86.c
+++ b/arch/x86/kvm/x86.c

```

```
@@ -9179,30 +9179,9 @@ int kvm_arch_vcpu_precreate(struct kvm *kvm, unsigned int id)
return 0;
}
```

```
-struct kvm_vcpu *kvm_arch_vcpu_create(struct kvm *kvm,
-                                     unsigned int id)
+int kvm_arch_vcpu_create(struct kvm_vcpu *vcpu)
{
-    struct kvm_vcpu *vcpu;
-    int r;
-
-    vcpu = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL_ACCOUNT);
-    if (!vcpu)
-    ...
```

This meets all our criteria except for the one we didn't initially notice - the function name in the conflict is different: `kvm_arch_vcpu_create` .vs. `svm_create_vcpu`

There is at least one more piece to the puzzle. **We need to account for that name change.**

but, it appears, **we may need to add `e529ef66e6b5` to our patch list.**

a9dd6f09d7e5, KVM: x86: Allocate vcpu struct in common x86 code, v5.10-rc1, 10/25/20

- oh, this looks very useful and it has the **right function name** (i.o.w this is after the, as yet unknown, rename patch.

```
@@ -2197,22 +2197,15 @@ static struct kvm_vcpu *svm_create_vcpu(struct kvm *kvm,
unsigned int id)
struct page *nested_msrpm_pages;
int err;
```

```
-    BUILD_BUG_ON_MSG(offsetof(struct vcpu_svm, vcpu) != 0,
-                      "struct kvm_vcpu must be at offset 0 for arch usercopy region");
-
-    svm = kmem_cache_zalloc(kvm_vcpu_cache, GFP_KERNEL_ACCOUNT);
-    if (!svm) {
-        err = -ENOMEM;
-        goto out;
-    }
-    vcpu = &svm->vcpu;
+    BUILD_BUG_ON(offsetof(struct vcpu_svm, vcpu) != 0);
+    svm = to_svm(vcpu);
-    ...
```

it appears, **we may need to add `a9dd6f09d7e5` to our patch list.**

at this point we might just add the two identified patches and restart the backport

if pre existing conflict resolutions have not been invalidated the previous resolution is re-used.

a previous resolution can be re-used if conflicts from previous attempt to cherry-pick patch P are identical to current cherry pick

but let's see what we can learn from the big hammer

cherry-pick --continue - after all conflicts resolved

note: The code to update the "commit <sha> downstream" line in the commit message is not there yet so you'll have to refer to the patch object and do it manually if needed.

state['all_patches']][0]

- {'sha1': '4543bdc08857', ..., 'subject': 'KVM: Introduce kvm_vcpu_destroy()', 'tag': 'v5.10-rc1', 'tag_date': '10/25/20', 'order_in_release': -1, 'downstream': '81dab744b886', ... }
- enter choice(<enter> to exit menu): 1

@@how_short_git_log

1: 81dab744b886 Sean Christopherson : KVM: Introduce kvm_vcpu_destroy()

A detailed look at "commits touching both modified files"

- we already know we need to add at least 2 patches and a third which we haven't identified yet
- let's see how many patches are hitting this file: arch/x86/kvm/svm.c.
- Menu option #7 shows all of the commits touching "both modified" files

Unmerged paths:

(use "git add <file>..." to mark resolution)

both modified: arch/x86/kvm/svm.c

both modified: arch/x86/kvm/vmx/vmx.c

both modified: arch/x86/kvm/x86.c

- Conveniently svm.c is first in line but there are 50 patches between the target we're backporting to (v5.4.32 and 987b2594ed5d in v5.10-rc1
0, 50896de4be7e, KVM: x86: always expose VIRT_SSBD to guests, v5.10-rc1, 10/25/20, unapplied
1, 05402f64544f, x86: KVM: svm: don't pretend to advance RIP in case wrmsr_interception() results in #GP, v5.10-rc1, 10/25/20, unapplied
2, f8ea7c6049d5, x86: kvm: svm: propagate errors from skip_emulated_instruction(), v5.10-rc1, 10/25/20, unapplied
3, 02d4160fbd76, x86: KVM: add xsetbv to the emulator, v5.10-rc1, 10/25/20, unapplied
4, c4762fdab55c, x86: KVM: svm: remove hardcoded instruction length from intercepts, v5.10-rc1, 10/25/20, unapplied
5, e7134c1bb5af, x86: KVM: svm: eliminate weird goto from vmrun_interception(), v5.10-rc1, 10/25/20, unapplied
6, c8e16b78c614, x86: KVM: svm: eliminate hardcoded RIP advancement from vmrun_interception(), v5.10-rc1, 10/25/20, unapplied
7, 12b58f4ed2a1, KVM: Assert that struct kvm_vcpu is always as offset zero, v5.10-rc1, 10/25/20, unapplied
8, fdcf75621375, KVM: x86: Disable posted interrupts for non-standard IRQs delivery modes, v5.10-rc1, 10/25/20, unapplied
9, 4f75bcc33221, KVM: X86: Tune PLE Window tracepoint, v5.10-rc1, 10/25/20, unapplied
10, f20935d85a23, KVM: x86: Refactor up kvm_{g,s}et_msr() to simplify callers, v5.10-rc1, 10/25/20, unapplied
11, 1edce0a9eb23, KVM: x86: Add kvm_emulate_{rd,wr}msr() to consolidate VXM/SVM code, v5.10-rc1, 10/25/20, unapplied

- 12, 7396d337cfad, KVM: x86: Return to userspace with internal error on unexpected exit reason, v5.10-rc1, 10/25/20, unapplied
- 13, a061985b81a2, x86: KVM: svm: Fix a check in nested_svm_vmrund(), v5.10-rc1, 10/25/20, unapplied
- 14, 4b9852f4f389, KVM: x86: Fix INIT signal handling in various CPU states, v5.10-rc1, 10/25/20, unapplied
- 15, fe38bd686207, Merge tag 'for-linus' of git://git.kernel.org/pub/scm/virt/kvm/kvm, v5.10-rc1, 10/25/20, unapplied
- 16, 956e255c59a5, KVM: x86: svm: remove unneeded nested_enable_evmcs() hook, v5.10-rc1, 10/25/20, unapplied
- 17, a6c6ed1e8106, KVM: x86: Don't attempt VMWare emulation on #GP with non-zero error code, v5.10-rc1, 10/25/20, unapplied
- 18, 42cbf06872cc, KVM: x86: Move #GP injection for VMware into x86_emulate_instruction(), v5.10-rc1, 10/25/20, unapplied
- 19, 738fece46dc5, KVM: x86: Exit to userspace on emulation skip failure, v5.10-rc1, 10/25/20, unapplied
- 20, 1051778f6e1e, KVM: x86: Handle emulation failure directly in kvm_task_switch(), v5.10-rc1, 10/25/20, unapplied
- 21, 60fc3d02d5b8, KVM: x86: Remove emulation_result enums, EMULATE_{DONE,FAIL,USER_EXIT}, v5.10-rc1, 10/25/20, unapplied
- 22, 1957aa63be53, KVM: VMX: Handle single-step #DB for EMULTYPE_SKIP on EPT misconfig, v5.10-rc1, 10/25/20, unapplied
- 23, 0cb8410b90e7, kvm: svm: Intercept RDPRU, v5.10-rc1, 10/25/20, unapplied
- 24, e3b9a9e147db, KVM: SVM: Serialize access to the SEV ASID bitmap, v5.10-rc1, 10/25/20, unapplied
- 25, 83af5e65a895, KVM: SVM: Guard against DEACTIVATE when performing WBINVD/DF_FLUSH, v5.10-rc1, 10/25/20, unapplied
- 26, 0fc5deae03a2, KVM: SVM: Remove unneeded WBINVD and DF_FLUSH when starting SEV guests, v5.10-rc1, 10/25/20, unapplied
- 27, 33af3a7ef9e6, KVM: SVM: Reduce WBINVD/DF_FLUSH invocations, v5.10-rc1, 10/25/20, unapplied
- 28, 34059c257010, KVM: x86: Fold decache_cr3() into cache_reg(), v5.10-rc1, 10/25/20, unapplied
- 29, 2cf9af0b5668, kvm: x86: Modify kvm_x86_ops.get_enable_apicv() to use struct kvm parameter, v5.10-rc1, 10/25/20, unapplied
- 30, 7204160eb780, KVM: x86: Introduce vcpu->arch.xsaves_enabled, v5.10-rc1, 10/25/20, unapplied
- 31, 312a1c87798e, KVM: SVM: Use wrmsr for switching between guest and host IA32_XSS on AMD, v5.10-rc1, 10/25/20, unapplied
- 32, 139a12cfe1a0, KVM: x86: Move IA32_XSS-swapping on VM-entry/VM-exit to common x86 code, v5.10-rc1, 10/25/20, unapplied
- 33, 52297436199d, kvm: svm: Update svm_xsaves_supported, v5.10-rc1, 10/25/20, unapplied
- 34, 5c94ac5d0f9e, KVM: SVM: Fix potential wrong physical id in avic_handle_ldr_update, v5.10-rc1, 10/25/20, unapplied
- 35, 9167ab799362, KVM: vmx, svm: always run with EFER.NXE=1 when shadow paging is active, v5.10-rc1, 10/25/20, unapplied
- 36, 3dcb2a3fa5a0, KVM: retpolines: x86: eliminate retpoline from svm.c exit handlers, v5.10-rc1, 10/25/20, unapplied

37, 49d654d85f85, KVM: SVM: Remove check if APICv enabled in SVM update_cr8_intercept() handler, v5.10-rc1, 10/25/20, unapplied

38, 46f4f0aabc61, Merge branch 'kvm-tsx-ctrl' into HEAD, v5.10-rc1, 10/25/20, unapplied

39, c1de0f25221c, KVM x86: Move kvm cpuid support out of svm, v5.10-rc1, 10/25/20, unapplied

40, ac8ef992cd02, KVM: X86: Drop KVM_APIC_SHORT_MASK and KVM_APIC_DEST_MASK, v5.10-rc1, 10/25/20, unapplied

41, 1e9e2622a149, KVM: VMX: FIXED+PHYSICAL mode single target IPI fastpath, v5.10-rc1, 10/25/20, unapplied

42, 96be4e069c93, KVM: x86: Drop special XSAVE handling from guest_cpuid_has(), v5.10-rc1, 10/25/20, unapplied

43, 87382003e355, KVM: x86: Refactor and rename bit() to feature_bit() macro, v5.10-rc1, 10/25/20, unapplied

44, 52918ed5fcf0, KVM: SVM: Override default MMIO mask if memory encryption is enabled, v5.10-rc1, 10/25/20, unapplied

45, 6441fa6178f5, KVM: x86: avoid incorrect writes to host MSR_IA32_SPEC_CTRL, v5.10-rc1, 10/25/20, unapplied

46, 7f27179a88a6, KVM: SVM: Use direct vcpu pointer during vCPU create/free, v5.10-rc1, 10/25/20, unapplied

47, a9dd6f09d7e5, KVM: x86: Allocate vcpu struct in common x86 code, v5.10-rc1, 10/25/20, unapplied

48, fc6e2a1845ab, KVM: x86: Move FPU allocation to common x86 code, v5.10-rc1, 10/25/20, unapplied

49, 987b2594ed5d, KVM: x86: Move kvm_vcpu_init() invocation to common code, v5.10-rc1, 10/25/20, cherry-pick active

- **at this point you have a decision to make among a few choices:**
 - **fine iterative approach**
 - **add the patches** we've identified
 - use the **restart_backport** menu item which will
 - abort the current cherry pick if any
 - increment the branch number
 - this way you can get back to the state before the most recent cherry-pick failed.
 - reset the incremented branch to the base commit you're backporting your patch list to
 - **coarse iterative, speculative approach=**
 - quickly review the patches and listen to the warning bells that sound if they touch files beyond svm.c(in our example)
 - such an expansion of scope is not unexpected or bad but you probably want to make a note of it and the current pickle file number if you need to go back to the patch list before a bunch of files are added
 - anything outside kvm is probably a danger siren and needs to be
 - thought about
 - noted
 - possibly discussed in team
 - possibly communicated to management if there's an unexpected and significant scope expansion
 - add hoc conflict resolution resulting in large differences with the upstream patches
 - customers will rebel over this if they notice.
- **closing notes**
 - either approach, if done correctly, will result in all the changes you need to have the first patch of your initial patch apply cleanly
 - this, in effect, means you'll have the same kvm, with respect to the interfaces used by the first target patch, as was present when target patch 1 was applied in the initial release that it appeared in. (sorry for the complexity of that sentence, but it's important)
 - this does **not** mean you'll have the same kvm just the subset that your target patch1 needs